utfcode
utf8.sty 3.10 UTF-8 input encoding 13.06.2000
scanner for code UTF-8 installed.

Machine Learning Practicum: Linear & Logistic Regression Worksheets

Department of Computer Science & Data Science

Instructions: Solve the following 20 lab tasks using Python (`pandas`, `numpy`, `scikit-learn`, `matplotlib`). Ensure all input matrices X are formatted as 2D arrays or DataFrames.

1. Linear Regression Lab Tasks

1.1 Task 1: Single Feature House Price Prediction

Scenario: Predict house price (y, in \$100k) based on living area (X, in 1000 sq ft).

```
import pandas as pd
df1 = pd.DataFrame({
  'SqFt_1000': [1.2, 1.5, 1.8, 2.1, 2.4, 2.7, 3.0, 3.3],
  'Price_100k': [2.5, 2.9, 3.4, 3.8, 4.2, 4.8, 5.1, 5.5]
})
```

Tasks:

1. Fit a `LinearRegression` model predicting `Price_100k` from `SqFt_1000`.
2. Report the model intercept (β_0) and slope (β_1).
3. Predict the price of a 2,200 sq ft house ($X = 2.2$).

1.2 Task 2: Multi-Feature Exam Performance

Scenario: Predict exam score (y, 0–100) using study hours (X_1) and sleep hours (X_2).

```
df2 = pd.DataFrame({
  'Study_Hours': [2, 3, 4, 5, 6, 7, 8, 9],
  'Sleep_Hours': [8, 7, 6, 8, 7, 6, 8, 7],
  'Exam_Score':  [55, 62, 68, 77, 83, 88, 95, 98]
})
```

Tasks:

1. Train a multiple linear regression model.
2. Interpret the coefficient for `Study_Hours` holding `Sleep_Hours` constant.
3. Predict the score for a student with 6 hours of study and 8 hours of sleep.

1.3 Task 3: Salary Estimation by Experience

Scenario: Predict annual software engineer salary (y, in \$1,000s) based on years of experience (X).

```
df3 = pd.DataFrame({
  'Years_Exp': [1, 2, 3, 4, 5, 6, 7, 8],
  'Salary_1k': [50, 56, 63, 70, 78, 85, 94, 102]
})
```

Tasks:

1. Calculate R^2 score and Mean Squared Error (MSE).
2. Plot the raw data points and the fitted regression line using `matplotlib`.

1.4 Task 4: Vehicle Depreciation Analysis

Scenario: Predict used car price (y, in \$1,000s) from vehicle age (X_1, years) and odometer reading (X_2, in 10k miles).

```
df4 = pd.DataFrame({
  'Age_Years': [1, 2, 3, 4, 5, 6, 7, 8],
  'Miles_10k': [1.0, 2.2, 3.5, 4.8, 5.5, 7.0, 8.2, 9.5],
  'Price_1k':  [24, 21, 18, 15, 13, 10, 8, 5]
})
```

Tasks:

1. Fit a multiple linear regression model.
2. Identify which feature has a stronger negative impact on price using standardized coefficients (`StandardScaler`).

1.5 Task 5: Utility Bill Modeling

Scenario: Estimate winter heating bill (y, USD) from square footage (X_1), average outdoor temperature (X_2, °F), and number of occupants (X_3).

```
df5 = pd.DataFrame({
  'SqFt':       [1200, 1500, 1800, 2000, 2200, 1400, 1600, 1900],
  'Avg_Temp_F': [35, 40, 45, 50, 55, 38, 42, 48],
  'Occupants':  [2, 3, 2, 4, 3, 1, 2, 4],
  'Bill_USD':   [150, 165, 170, 210, 220, 145, 160, 200]
})
```

Tasks:

1. Compute the model's R^2 and Adjusted R^2.
2. Explain why Adjusted R^2 is preferred when working with multiple features.

1.6 Task 6: Crop Yield Optimization

Scenario: Predict crop yield (y, tons/acre) based on nitrogen fertilizer applied (X_1, kg/acre) and rainfall (X_2, mm).

```
df6 = pd.DataFrame({
  'Fertilizer_kg': [10, 20, 30, 40, 50, 60, 70, 80],
  'Rainfall_mm':   [100, 110, 120, 130, 140, 150, 160, 170],
  'Yield_Tons':    [2.1, 2.8, 3.4, 4.0, 4.5, 5.1, 5.6, 6.0]
})
```

Tasks:

1. Fit linear regression and calculate residual values $e_i = y_i - \hat{y}_i$.
2. Plot predicted values $\hat{y}$ vs. residuals e to check for homoscedasticity.

1.7 Task 7: Tech Hardware Pricing

Scenario: Predict laptop price (y, USD) using RAM (X_1, GB) and SSD Storage (X_2, GB).

```
df7 = pd.DataFrame({
  'RAM_GB':    [8, 8, 16, 16, 32, 32, 16, 64],
  'SSD_GB':    [256, 512, 512, 1024, 1024, 2048, 256, 2048],
  'Price_USD': [500, 620, 850, 1100, 1450, 1900, 780, 2400]
})
```

Tasks:

1. Fit a linear regression model and write down the prediction equation.
2. Estimate the price for a laptop with 32 GB RAM and 512 GB SSD.

1.8 Task 8: Digital Marketing ROI

Scenario: Predict monthly sales revenue (y, in \$1,000s) from ad spend (X_1, in \$1,000s) and social media posts (X_2).

```
df8 = pd.DataFrame({
  'Ad_Spend_1k':  [1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0, 4.5],
  'Social_Posts': [10, 15, 20, 22, 28, 30, 35, 40],
  'Revenue_1k':   [12, 16, 21, 25, 30, 34, 40, 44]
})
```

Tasks:

1. Train the model and report Mean Squared Error (MSE).
2. Calculate the feature correlation matrix using `df8.corr()`.

1.9 Task 9: Delivery Logistics

Scenario: Predict delivery time (y, minutes) based on route distance (X_1, miles) and drop-off stops (X_2).

```
df9 = pd.DataFrame({
  'Distance_Miles': [2, 4, 6, 8, 10, 12, 14, 16],
  'Stops':          [1, 2, 2, 3, 4, 4, 5, 6],
  'Time_Min':       [15, 25, 32, 42, 53, 60, 71, 82]
})
```

Tasks:

1. Fit a multiple linear regression model.
2. Determine the expected delay per additional delivery stop holding distance constant.

1.10 Task 10: Restaurant Revenue Metrics

Scenario: Predict daily revenue (y, in \$1,000s) based on seating capacity (X_1) and daily foot traffic (X_2).

```
df10 = pd.DataFrame({
  'Seats':        [30, 40, 50, 60, 70, 80, 90, 100],
  'Foot_Traffic': [150, 200, 280, 320, 400, 450, 520, 600],
  'Revenue_1k':   [2.1, 2.9, 3.8, 4.5, 5.4, 6.2, 7.1, 8.0]
})
```

Tasks:

1. Fit `LinearRegression` and evaluate using R^2.
2. Compare actual vs. predicted revenue for all 8 observations.

2. Logistic Regression Lab Tasks

2.1 Task 1: Single Feature Pass/Fail Prediction

Scenario: Predict whether a student passes an exam ($y \in \{0, 1\}$) based on study hours (X).

```
clf1 = pd.DataFrame({
  'Study_Hours': [0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0, 4.5, 5.0],
  'Passed':      [0,   0,   0,   0,   0,   1,   0,   1,   1,   1]
})
```

Tasks:

1. Fit a `LogisticRegression` model predicting `Passed`.
2. Calculate the decision boundary where $P(\text{Passed}) = 0.5$.
3. Predict pass probability for a student who studied 2.8 hours.

2.2 Task 2: Credit Risk Assessment

Scenario: Predict loan approval ($1 = \text{Approved}, 0 = \text{Denied}$) based on credit score (X_1) and annual income (X_2, in \$1,000s).

```
clf2 = pd.DataFrame({
  'Credit_Score': [580, 610, 640, 660, 700, 720, 750, 800],
  'Income_1k':    [35,  40,  45,  60,  55,  80,  95,  110],
  'Approved':     [0,   0,   0,   0,   1,   1,   1,   1]
})
```

Tasks:

1. Train a logistic regression classifier.
2. Output predicted probabilities for all samples using `model.predict_proba(X)`.
3. Calculate training set accuracy score.

2.3 Task 3: Email Spam Detection

Scenario: Classify an email as spam (1) or not spam (0) based on word count (X_1) and link count (X_2).

```
clf3 = pd.DataFrame({
  'Word_Count': [50, 120, 450, 800, 30, 200, 600, 90],
  'Link_Count': [0,  1,   8,   12,  0,  2,   10,  5],
  'Is_Spam':    [0,  0,   1,   1,   0,  0,   1,   1]
})
```

Tasks:

1. Fit logistic regression and display model intercept and coefficients.
2. Generate a confusion matrix using `sklearn.metrics.confusion_matrix`.

2.4 Task 4: Heart Disease Screening

Scenario: Predict high heart disease risk (1) vs. low risk (0) using age (X_1) and cholesterol levels (X_2, mg/dL).

```
clf4 = pd.DataFrame({
  'Age':         [30, 40, 45, 50, 55, 60, 65, 70],
  'Cholesterol': [180, 195, 210, 240, 220, 260, 280, 300],
  'High_Risk':   [0,   0,   0,   1,   0,   1,   1,   1]
})
```

Tasks:

1. Fit the model and compute Precision, Recall, and F1-score.
2. Identify any false positive or false negative predictions.

2.5 Task 5: Customer Churn Prediction

Scenario: Predict customer churn ($1 = \text{Yes}, 0 = \text{No}$) using monthly charge ($X_1$, USD) and tenure ($X_2$, months).

```
clf5 = pd.DataFrame({
  'Monthly_Charge': [25, 35, 65, 85, 45, 95, 105, 30],
  'Tenure_Months':  [24, 36, 6,  2,  18, 4,  1,   48],
  'Churned':        [0,  0,  1,  1,  0,  1,  1,   0]
})
```

Tasks:

1. Train `LogisticRegression`.
2. Convert log-odds coefficients to Odds Ratios using $\text{OR} = e^{\beta}$.
3. Interpret how tenure affects churn probability.

2.6 Task 6: University Admission Model

Scenario: Predict admission ($1 =$ Admitted, $0 =$ Rejected) using GPA (X_1, 0–4.0 scale) and SAT score (X_2, 400–1600).

```
clf6 = pd.DataFrame({
  'GPA':      [2.8, 3.1, 3.4, 3.5, 3.7, 3.8, 3.9, 4.0],
  'SAT':      [1050, 1120, 1200, 1280, 1340, 1420, 1490, 1550],
  'Admitted': [0,    0,    0,    1,    1,    1,    1,    1]
})
```

Tasks:

1. Fit model and plot predicted probability curve against GPA.
2. Predict admission status for a student with GPA $= 3.3$ and SAT $= 1220$.

2.7 Task 7: Employee Promotion Likelihood

Scenario: Predict promotion status ($1 =$ Promoted, $0 =$ Not Promoted) based on performance rating (X_1, 1–5) and tenure (X_2, years).

```
clf7 = pd.DataFrame({
  'Rating':   [2, 3, 3, 4, 4, 5, 5, 5],
  'Years':    [1, 2, 5, 3, 6, 2, 4, 7],
  'Promoted': [0, 0, 0, 0, 1, 1, 1, 1]
})
```

Tasks:

1. Fit `LogisticRegression` and print `classification_report`.
2. Evaluate overall prediction accuracy on the dataset.

2.8 Task 8: E-Commerce Purchase Intent

Scenario: Predict if a visitor buys an item (1) or leaves (0) using time on site (X_1, seconds) and pages viewed (X_2).

```
clf8 = pd.DataFrame({
  'Time_Sec':  [20, 45, 90, 150, 210, 300, 420, 600],
  'Pages':     [1,  2,  3,  4,   5,   7,   8,   12],
  'Purchased': [0,  0,  0,  0,   1,   1,   1,   1]
})
```

Tasks:

1. Standardize features using `StandardScaler` prior to fitting.
2. Compute ROC-AUC score using `roc_auc_score`.

2.9 Task 9: Real Estate Deal Closure

Scenario: Classify if an offer is accepted (1) or rejected (0) based on discount off list price (X_1, %) and days on market (X_2).

```
clf9 = pd.DataFrame({
  'Discount_Pct': [2,  4,  6,  8,  10, 12, 15, 18],
  'Days_Market':  [12, 20, 45, 60, 30, 90, 80, 110],
  'Accepted':     [1,  1,  1,  0,  1,  0,  0,  0]
})
```

Tasks:

1. Fit a logistic regression model.
2. Identify which feature negatively impacts acceptance probability more strongly.

2.10 Task 10: Botanical Plant Survival

Scenario: Predict plant survival ($1 = \text{Survived}, 0 = \text{Died}$) based on daily sunlight (X_1, hours) and daily water (X_2, mL).

```
clf10 = pd.DataFrame({
  'Sunlight_Hrs': [2, 3, 4, 5, 6, 7, 8, 9],
  'Water_mL':     [50, 80, 100, 150, 200, 250, 300, 350],
  'Survived':     [0,  0,  0,   1,   1,   1,   1,   1]
})
```

Tasks:

1. Fit `LogisticRegression`.
2. Predict survival class and probability for a plant receiving 4.5 hours of sunlight and 120 mL water.